



Instituto Infnet

2º Ciclo - Fundamentos do Processamento de Dados

Python para Processamento de Dados [26E2_4]

Exercício

Crie um algoritmo que compare duas lista e determine:

- os valores comuns às duas listas
- os valores que só existem na primeira lista
- os valores que existem apenas a segunda lista
- uma lista com os elementos não repetidos das duas lista

```
tamanho_lista_1 = int(input('Digite o tamanho da primeira lista:'))
tamanho_lista_2 = int(input('Digite o tamanho da segunda lista:'))

# criando as listas vazias
lista_1 = list()
lista_2 = list()

print(f'Digite os {tamanho_lista_1} valores da primeira lista:')
for i in range(tamanho_lista_1):
    numero = int(input(f'Digite {i+1}o valor:'))
    lista_1.append(numero)

print('Primeira lista:', lista_1)

print(f'Digite os {tamanho_lista_2} valores da segunda lista:')
for i in range(tamanho_lista_2):
    numero = int(input(f'Digite {i+1}o valor:'))
    lista_2.append(numero)

print('Segunda lista:', lista_2)
```

```
Digite os 3 valores da primeira lista:
Primeira lista: [2, 2, 2]
Digite os 3 valores da segunda lista:
Segunda lista: [2, 2, 2]
```

```

print('verificando os valores comuns às duas listas')

# forma 1: iterando nas listas
numeros_comuns = list()
for valor_lista_1 in lista_1:
    if valor_lista_1 in lista_2:
        if valor_lista_1 not in numeros_comuns:
            numeros_comuns.append(valor_lista_1)

if len(numeros_comuns) > 0:
    print('Os numeros comuns as listas digitadas:', numeros_comuns)
else:
    print('Nao existe numero comuns as listas digitadas.')

# forma 2: utilizando conjuntos
numeros_comuns_conjuntos = set(lista_1) & set(lista_2)

if len(numeros_comuns_conjuntos) > 0:
    print('Os numeros comuns as listas digitadas:', numeros_comuns_conjuntos)
else:
    print('Nao existe numero comuns as listas digitadas.')

```

```

verificando os valores comuns às duas listas
Os numeros comuns as listas digitadas: [2]
Os numeros comuns as listas digitadas: {2}

```

```

print('verificando os valores que só existem na primeira listas')

valores_primeira_lista = set(lista_1) - set(lista_2)
print('Valores que só existem na primeira listas:', valores_primeira_lista)

```

```

verificando os valores que só existem na primeira listas
Valores que só existem na primeira listas: set()

```

```

print('verificando os valores que existem apenas a segunda lista')
valores_segunda_lista = set(lista_2) - set(lista_1)
print('Valores que só existem na primeira listas:', valores_segunda_lista)

```

```

verificando os valores que existem apenas a segunda lista
Valores que só existem na primeira listas: set()

```

```

print('verificando os elementos não repetidos das duas lista')
valores_lista = set(lista_1) | set(lista_2)
print('Os elementos não repetidos das duas lista:', valores_lista)

```

```

verificando os elementos não repetidos das duas lista
Os elementos não repetidos das duas lista: {2}

```

Módulos

Módulos são arquivos Python que reúnem código reutilizável, como funções, classes e variáveis, podendo ser importados e utilizados em outros programas. Eles são importantes para organizar melhor o código, facilitar a reutilização de funcionalidades e evitar conflitos de nomes em projetos maiores.

Conceitos Básicos

```

# importando modulos.

# como realizar requisicoes web com python?
# eh necessario obter um codigo disponivel da biblioteca python que realize esse atividade.

import requests

```

```
response = requests.get('https://www.infnet.edu.br/')
type(response)
```

```
requests.models.Response
```

```
response.status_code
```

```
200
```

```
# biblioteca para estrutura de dados tabular (dados retangular)
# pandas
import pandas as pd

df = pd.DataFrame([{'nome' : 'Gesiel Lopes', 'idade': 46}, {'nome' : 'Jonh Doe', 'idade': 42}, {'nome' : 'Marie Curie', 'idade': 52}])
df
```

	nome object	idade int64	
0	Gesiel Lopes	46	
1	Jonh Doe	42	
2	Marie Curie	52	

3 rows, 2 cols 10 / page << < Page 1 of 1 > >> [↓](#)

```
# Como utilizar a implementacao de um algoritmo de ML para realizar agrupamento?
```

```
import numpy as np
from sklearn.cluster import KMeans

X = np.array([[1, 2], [1, 4], [1, 0],
              [10, 2], [10, 4], [10, 0]])
```

```
kmeans = KMeans(n_clusters=2).fit(X)
kmeans.labels_
```

```
/root/venv/lib/python3.11/site-packages/sklearn/cluster/_kmeans.py:1416: FutureWarning: The default value of `n_init` will change from 10 to 1 in version 1.4. For now, it defaults to 10 if left unchanged.
  super()._check_params_vs_input(X, default_n_init=10)
```

```
array([0, 0, 0, 1, 1, 1], dtype=int32)
```

```
# importar todos modulos e submodulos de um modulo especifico
```

```
from sklearn.cluster import *
```

Módulos da Biblioteca Padrão

```
# math
```

```
import math
```

```
math.pi
```

```
3.141592653589793
```

```
# random
```

```
import random
```

```
random.random()
```

```
0.33724393582179646
```

```
# datetime
```

```
import datetime
```

```
datetime.date.today()
```

```
datetime.date(2026, 5, 14)
```

```
# os e sys
```

```
import os
```

```
import sys
```

```
print(os.environ)
```

```
print(sys.platform)
```

```
print(sys.version)
```

```
environ({'KUBERNETES_SERVICE_PORT': '443', 'PLOTLY_RENDERER': 'colab', 'KUBERNETES_PORT': 'tcp://10.96.0.1:443', 'LANGUAGE': 'en_
linux
```

```
3.11.14 (main, Feb 20 2026, 09:55:14) [GCC 12.2.0]
```

Instalando e Gerenciando Pacotes

Python tem um sistema de pacotes rico, permitindo instalar módulos externos.

```
!pip install PuLP
```

```
Collecting PuLP
```

```
  Downloading pulp-3.3.1-py3-none-any.whl.metadata (8.3 kB)
```

```
Downloading pulp-3.3.1-py3-none-any.whl (16.4 MB)
```

```
----- 16.4/16.4 MB 97.4 MB/s eta 0:00:00
```

```
Installing collected packages: PuLP
```

```
Successfully installed PuLP-3.3.1
```

```
[notice] A new release of pip is available: 24.0 -> 26.1.1
```

```
[notice] To update, run: pip install --upgrade pip
```

```
from pulp import *
```

```
prob = LpProblem("myProblem", LpMinimize)
x = prob.add_variable("x", 0, 3)
y = prob.add_variable("y", cat="Binary")
```

```
prob += x + y <= 2
prob += -4*x + y
```

```
status = prob.solve()
```

```
status
```

Welcome to the CBC MILP Solver

Version: 2.10.3

Build Date: Dec 15 2019

command line - /root/venv/lib/python3.11/site-packages/pulp/apis/./solverdir/cbc/linux/i64/cbc /tmp/2295b0f5a1d148ee99905b183d59

At line 2 NAME MODEL

At line 3 ROWS

At line 6 COLUMNS

At line 13 RHS

At line 15 BOUNDS

At line 18 ENDDATA

Problem MODEL has 1 rows, 2 columns and 2 elements

Coin0008I MODEL read with 0 errors

Option for timeMode changed from cpu to elapsed

Continuous objective value is -8 - 0.00 seconds

Cgl0004I processed model has 0 rows, 0 columns (0 integer (0 of which binary)) and 0 elements

Cbc3007W No integer variables - nothing to do

Cuts at root node changed objective from -8 to -1.79769e+308

Probing was tried 0 times and created 0 cuts of which 0 were active after adding rounds of cuts (0.000 seconds)

Gomory was tried 0 times and created 0 cuts of which 0 were active after adding rounds of cuts (0.000 seconds)

Knapsack was tried 0 times and created 0 cuts of which 0 were active after adding rounds of cuts (0.000 seconds)

Clique was tried 0 times and created 0 cuts of which 0 were active after adding rounds of cuts (0.000 seconds)

MixedIntegerRounding2 was tried 0 times and created 0 cuts of which 0 were active after adding rounds of cuts (0.000 seconds)

FlowCover was tried 0 times and created 0 cuts of which 0 were active after adding rounds of cuts (0.000 seconds)

TwoMirCuts was tried 0 times and created 0 cuts of which 0 were active after adding rounds of cuts (0.000 seconds)

ZeroHalf was tried 0 times and created 0 cuts of which 0 were active after adding rounds of cuts (0.000 seconds)

Result - Optimal solution found

Objective value: -8.00000000

1

Aprofundamento

- [Documentação sobre módulos](#)
- [Índice de Pacotes Python \(PyPI\)](#)